

# Student Lab Manual

## Full-Stack Deployment Inside an Ubuntu Docker Container

Install Git, .NET SDK/Runtime, Node.js, Angular CLI, Nginx; pull source code from GitHub; deploy ASP.NET Core API and Angular app

Lab Duration: 3 to 5 hours

Target Audience: Beginners learning Linux containers, application deployment, Nginx hosting, and full-stack deployment basics.

Platform Assumption: Docker Desktop running on Windows. Commands are written for Windows PowerShell or Command Prompt on the host, and Bash inside the Ubuntu container.

**Cover / Completion Screenshot Placeholder**

**Paste screenshot here**

## Lab Overview

In this lab, students will use an Ubuntu Linux Docker container like a small Linux server. Inside the container, they will install required tools, clone two GitHub repositories, run a .NET backend API, build an Angular frontend, and serve the Angular app through Nginx. Nginx will also reverse proxy API calls to the .NET application.

The Angular application will be accessible from the local system browser using `http://localhost:8080`. The backend API will run inside the same container and will be reached through Nginx using `/api` routes.

```
Host Machine / Browser
|
| http://localhost:8080
v
Docker Port Mapping: 8080 -> 80
|
v
Ubuntu Container
|
+-- Nginx on port 80
|   +-- Serves Angular static files from /var/www/html/angular
|   +-- Reverse proxies /api to ASP.NET Core API
|
+-- ASP.NET Core API running on http://localhost:5000
|
+-- Git, .NET SDK, Node.js, npm, Angular CLI installed
```

| Placeholder                                 | Value to Fill                                                                                                 |
|---------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Backend API GitHub Repository URL           | <a href="https://github.com/114-Vaibhav/demo-dotnet-api">https://github.com/114-Vaibhav/demo-dotnet-api</a>   |
| Angular GitHub Repository URL               | <a href="https://github.com/114-Vaibhav/demo-angular-app">https://github.com/114-Vaibhav/demo-angular-app</a> |
| Backend project folder name after git clone | Demo-dotnet-api                                                                                               |
| Angular project folder name after git clone | Demo-angular-app                                                                                              |
| Backend project .csproj file name           | DemoApi.csproj                                                                                                |
| Angular build output folder under dist      | Demo-angular-app                                                                                              |

## Important Notes for Students

- This lab intentionally installs software manually inside the Ubuntu container. This is useful for learning Linux server administration inside containers.
- For production, we normally write a Dockerfile instead of manually installing tools and committing containers.
- Do not close the terminal running the container unless instructed.
- If a command asks for confirmation, use `-y` where shown or type `y` and press Enter.
- Replace placeholder values like `<DOTNET_API_REPO_URL>` and `<ANGULAR_REPO_URL>` with your actual GitHub repository URLs.

## Step 1: Pull Ubuntu Linux Image

**Objective:** Download the Ubuntu Linux image to your local Docker environment.

### Commands:

```
docker pull ubuntu:24.04
```

```
docker images
```

### Command Explanation:

Vaibhav Gupta (vgupta1@presidio.com)

- docker pull downloads an image from Docker Hub.
- ubuntu:24.04 means Ubuntu version 24.04.
- docker images lists all images available locally.

### Expected Output / Result:

You should see ubuntu with tag 24.04 in the image list.

### Verification:

Confirm that the ubuntu:24.04 image appears in the output of docker images.

```
Last login: Mon Jun 22 12:17:54 on ttys000
/dev/fd/12:25: command not found: compdef
vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 42 June 23 % docker pull ubuntu:24.04

docker images

24.04: Pulling from library/ubuntu
Digest: sha256:786a8b558f7be160c6c8c4a54f9a57274f3b4fb1491cf65146521ae77ff1dc54
Status: Image is up to date for ubuntu:24.04
docker.io/library/ubuntu:24.04

What's next:
_ View a summary of image vulnerabilities and recommendations → docker scout quickview ubuntu:24.04
```

| IMAGE                               | ID           | DISK USAGE | CONTENT SIZE | EXTRA |
|-------------------------------------|--------------|------------|--------------|-------|
| alpine:latest                       | 28bd5fe8b56d | 13.6MB     | 4.27MB       | U     |
| compose-postgres-api-lab-api:latest | aebc950f8bf  | 387MB      | 97MB         | U     |
| customer-api:1.0                    | 62593972ba26 | 350MB      | 88.3MB       | U     |
| dpage/pgadmin4:latest               | cefc4cc6b7d9 | 765MB      | 175MB        | U     |
| nginx:latest                        | 42f2d24ae18d | 259MB      | 64.3MB       | U     |
| node:22-alpine                      | e58326d0d441 | 229MB      | 58.5MB       |       |
| postgres:16                         | 081f1bc7bd5e | 663MB      | 165MB        | U     |
| postgres:latest                     | 29ee7bb30d80 | 671MB      | 167MB        | U     |
| ubuntu-nginx:v1                     | 588195ce8e26 | 240MB      | 68.2MB       | U     |
| ubuntu:24.04                        | 786a8b558f7b | 141MB      | 30.8MB       | U     |
| weather-ui:1.0                      | 399165a2f358 | 76.1MB     | 21.9MB       |       |
| weatherapi:1.0                      | aefb12c8db89 | 351MB      | 88.4MB       |       |
| weatherapi:single                   | 8cf0eb18e30e | 1.25GB     | 311MB        |       |

```
vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 42 June 23 %
```

Student Notes: Ubuntu image is up to date.

## Step 2: Start Ubuntu Container with Port Mapping

**Objective:** Create an Ubuntu container and expose ports so the host browser can access the app later.

### Commands:

```
docker run -it --name fullstack-ubuntu -p 8080:80 -p 5000:5000 ubuntu:24.04 bash
```

Vaibhav Gupta (vgupta1@presidio.com)

### Command Explanation:

- -it starts the container in interactive mode.
- --name fullstack-ubuntu assigns a readable container name.
- -p 8080:80 maps host port 8080 to container port 80 for Nginx.
- -p 5000:5000 maps host port 5000 to container port 5000 for direct API testing.
- bash opens a Linux shell inside the container.

### Expected Output / Result:

You should see a Linux prompt similar to root@container-id:/#.

### Verification:

Run pwd and whoami. Expected current user is root.

```
vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 42 June 23 % docker run -it --name fullstack-ubuntu -p 8080:80 -p 5000:5000 ubuntu:24.04 bash

What's next:
  Debug this container error with Gordon → docker ai "help me fix this container error"
docker: Error response from daemon: ports are not available: exposing port TCP 0.0.0.0:5000 -> 127.0.0.1:0: listen tcp 0.0.0.0:5000: bind: address already in use.

Run 'docker run --help' for more information
vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 42 June 23 % docker run -it --name fullstack-ubuntu -p 8080:80 -p 5000:5000 ubuntu:24.04 bash

What's next:
  Debug this container error with Gordon → docker ai "help me fix this container error"
docker: Error response from daemon: Conflict. The container name "/fullstack-ubuntu" is already in use by container "ffd4fa9e2ad66aefae4de95fdc3fbc5aa82" (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 42 June 23 % docker rm fullstack-ubuntu
fullstack-ubuntu
vaibhavgupta@FVFG905WQ05F-vaibhavgupta Day 42 June 23 % docker run -it --name fullstack-ubuntu -p 8080:80 -p 5000:5000 ubuntu:24.04 bash
root@d0d317c574c2:/#
```

Student Notes: In mac port 5000, used by AirPlay so need to turn off.

## Step 3: Update Ubuntu Package Repository

**Objective:** Refresh package information before installing software.

### Commands:

```
apt update
```

### Command Explanation:

- apt is Ubuntu package manager.
- apt update downloads the latest package index.
- This does not install software; it updates package metadata.

### Expected Output / Result:

Expected output includes Reading package lists... Done.

### Verification:

Vaibhav Gupta (vgupta1@presidio.com)

No error should appear at the end of the command.

```
root@d0d317c574c2:/# apt update
Get:1 http://ports.ubuntu.com/ubuntu-ports noble InRelease [256 kB]
Get:2 http://ports.ubuntu.com/ubuntu-ports noble-updates InRelease [126 kB]
Get:3 http://ports.ubuntu.com/ubuntu-ports noble-backports InRelease [126 kB]
Get:4 http://ports.ubuntu.com/ubuntu-ports noble-security InRelease [126 kB]
Get:5 http://ports.ubuntu.com/ubuntu-ports noble/universe arm64 Packages [19.0 MB]
Get:6 http://ports.ubuntu.com/ubuntu-ports noble/multiverse arm64 Packages [274 kB]
Get:7 http://ports.ubuntu.com/ubuntu-ports noble/restricted arm64 Packages [113 kB]
Get:8 http://ports.ubuntu.com/ubuntu-ports noble/main arm64 Packages [1776 kB]
Get:9 http://ports.ubuntu.com/ubuntu-ports noble-updates/universe arm64 Packages [2098 kB]
Get:10 http://ports.ubuntu.com/ubuntu-ports noble-updates/restricted arm64 Packages [2020 kB]
Get:11 http://ports.ubuntu.com/ubuntu-ports noble-updates/main arm64 Packages [1307 kB]
Get:12 http://ports.ubuntu.com/ubuntu-ports noble-updates/multiverse arm64 Packages [63.8 kB]
Get:13 http://ports.ubuntu.com/ubuntu-ports noble-backports/main arm64 Packages [48.9 kB]
Get:14 http://ports.ubuntu.com/ubuntu-ports noble-backports/multiverse arm64 Packages [671 B]
Get:15 http://ports.ubuntu.com/ubuntu-ports noble-backports/universe arm64 Packages [36.3 kB]
Get:16 http://ports.ubuntu.com/ubuntu-ports noble-security/universe arm64 Packages [1519 kB]
Get:17 http://ports.ubuntu.com/ubuntu-ports noble-security/main arm64 Packages [995 kB]
Get:18 http://ports.ubuntu.com/ubuntu-ports noble-security/multiverse arm64 Packages [59.5 kB]
Get:19 http://ports.ubuntu.com/ubuntu-ports noble-security/restricted arm64 Packages [1933 kB]
Fetched 31.9 MB in 7s (4883 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
6 packages can be upgraded. Run 'apt list --upgradable' to see them.
root@d0d317c574c2:/#
```

Student Notes: working fine, everything updated

## Step 4: Install Basic Utilities and Git

**Objective:** Install tools required for downloading packages and cloning GitHub repositories.

### Commands:

```
apt install -y git curl wget nano ca-certificates gnupg lsb-release software-properties-common
```

### Command Explanation:

- git is used to clone source code from GitHub.
- curl and wget are used to download files from the internet.
- nano is a beginner-friendly terminal editor.
- ca-certificates and gnupg help with secure package repositories.
- -y automatically confirms installation.

### Expected Output / Result:

Packages should be installed without errors.

### Verification:

Run `git --version` and `curl --version`.

```
Setting up gpg-wks-client (2.4.4-2ubuntu17.4) ...
Setting up python3 (3.12.3-0ubuntu2.1) ...
running python rtupdate hooks for python3.12...
running python post-rtupdate hooks for python3.12...
Setting up python3-six (1.16.0-4) ...
Setting up python3-pyparsing (3.1.1-1) ...
Setting up python3-gi (3.48.2-1) ...
Setting up python3-httplib2 (0.20.4-3) ...
Setting up python3-distro-info (1.7build1) ...
Setting up python3-cffi-backend:arm64 (1.16.0-2build1) ...
Setting up python3-blinker (1.7.0-1) ...
Setting up python3-pkg-resources (68.1.2-2ubuntu1.2) ...
Setting up python3-dbus (1.3.2-5build3) ...
Setting up python3-distro (1.9.0-1) ...
Setting up python3-jwt (2.7.0-1ubuntu0.1) ...
Setting up python3-apt (2.7.7ubuntu5.2) ...
Setting up python3-lazr.uri (1.0.6-3) ...
Setting up unattended-upgrades (2.9.1+nmu4ubuntu1) ...
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog based frontend cannot be used. at /usr/share/perl5/Debconf/FrontEnd/Dialog.pm line 79.)
debconf: falling back to frontend: Readline

Creating config file /etc/apt/apt.conf.d/20auto-upgrades with new version

Creating config file /etc/apt/apt.conf.d/50unattended-upgrades with new version
Created symlink /etc/systemd/system/multi-user.target.wants/unattended-upgrades.service → /usr/lib/systemd/system/unattended-upgrades.service.
Setting up python3-cryptography (41.0.7-4ubuntu0.4) ...
Setting up python3-wadllib (1.3.6-5) ...
Setting up networkd-dispatcher (2.2.4-4) ...
Created symlink /etc/systemd/system/multi-user.target.wants/networkd-dispatcher.service → /usr/lib/systemd/system/networkd-dispatcher.service.
Setting up python3-cauthlib (3.2.2-1) ...
Setting up python3-lazr.restfulclient (0.14.6-1) ...
Setting up python3-launchpadlib (1.11.0-6) ...
Created symlink /etc/systemd/user/timers.target.wants/launchpadlib-cache-clean.timer → /usr/lib/systemd/user/launchpadlib-cache-clean.timer.
Setting up python3-software-properties (0.99.49.4) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
Processing triggers for sgml-base (1.31) ...
Setting up polkitd (124-2ubuntu1.24.04.3) ...
Creating group 'polkitd' with GID 995.
Creating user 'polkitd' (User for polkitd) with UID 995 and GID 995.
invoke-rc.d: could not determine current runlevel
invoke-rc.d: policy-rc.d denied execution of reload.
start-stop-daemon: unable to stat /usr/libexec/polkitd (No such file or directory)
Setting up packagekit (1.2.8-2ubuntu1.5) ...
invoke-rc.d: could not determine current runlevel
invoke-rc.d: policy-rc.d denied execution of force-reload.
Failed to open connection to "system" message bus: Failed to connect to socket /run/dbus/system_bus_socket: No such file or directory
Created symlink /etc/systemd/user/sockets.target.wants/pk-debconf-helper.socket → /usr/lib/systemd/user/pk-debconf-helper.socket.
Setting up packagekit-tools (1.2.8-2ubuntu1.5) ...
Setting up software-properties-common (0.99.49.4) ...
Processing triggers for ca-certificates (20240601-24.04.1) ...
Updating certificates in /etc/ssl/certs...
0 added, 0 removed; done.
Running hooks in /etc/ca-certificates/update.d...
done.
Processing triggers for dbus (1.14.10-4ubuntu4.1) ...
root@d0d317c574c2:/#
```

```
root@d0d317c574c2:/# git --version
git version 2.43.0
root@d0d317c574c2:/# curl --version
curl 8.5.0 (aarch64-unknown-linux-gnu) libcurl/8.5.0 OpenSSL/3.0.13 zlib/1.3 brotli/1.1.0 zstd/1.5.5 libidn2/2.3.7 libpsl/0.21.2 (+libidn2/2.3.7) libssh/0.10.6/openssh/zlib nghttp2/1.59.0
enLDAP/2.6.10
Release-Date: 2023-12-06, security patched: 8.5.0-2ubuntu10.9
Protocols: dict file ftp ftps gopher gophers http https imap imaps ldap ldaps mqtt pop3s pop3s rtsp scp sftp smb smbs smtp smtps telnet tftp
Features: alt-svc AsynchDNS brotli GSS-API HSTS HTTP2 HTTPS-proxy IDN IPv6 Kerberos Largefile libz NTLM PSL SPNEGO SSL threadsafe TLS-SRP UnixSockets zstd
root@d0d317c574c2:/#
```

Student Notes: Installed successfully, asked for region and timezone.

## Step 5: Verify Git Installation

**Objective:** Confirm Git is available inside the Ubuntu container.

**Commands:**

```
git --version
```

**Command Explanation:**

- `git --version` displays the installed Git version.
- This proves the container can use Git commands.

**Expected Output / Result:**



Vaibhav Gupta (vgupta1@presidio.com)

Expected output: git version x.xx.x

### Verification:

Paste the git version screenshot.

```
root@d0d317c574c2:/# git --version
git version 2.43.0
root@d0d317c574c2:/# curl --version
curl 8.5.0 (aarch64-unknown-linux-gnu) libcurl/8.5.0 OpenSSL/3.0.13 zlib/1.3 brotli/1.1.0 zstd/1.5.5 libidn2/2.3.7 libpsl/0.21.2 (+libidn2/2.3.7) libssh/0.10.6/openssl/zlib nghttp2/1.59.0
enLDAP/2.6.10
Release-Date: 2023-12-06, security patched: 8.5.0-2ubuntu10.9
Protocols: dict file ftp ftps gopher gophers http https imap imaps ldap ldaps mqtt pop3 pop3s rtmp rtsp scp sftp smb smbs smtp smtps telnet tftp
Features: alt-svc AsynchDNS brotli GSS-API HSTS HTTP2 HTTPS-proxy IDN IPv6 Kerberos Largefile libz NTLM PSL SPNEGO SSL threadsafe TLS-SRP UnixSockets zstd
root@d0d317c574c2:/#
```

Student Notes: git installed

## Step 6: Install .NET 8 SDK and Runtime

**Objective:** Install .NET SDK so you can restore, build, publish, and run the ASP.NET Core application.

### Commands:

```
wget https://packages.microsoft.com/config/ubuntu/24.04/packages-microsoft-prod.deb -O packages-microsoft-prod.deb
```

```
dpkg -i packages-microsoft-prod.deb
```

```
apt update
```

```
apt install -y dotnet-sdk-8.0 aspnetcore-runtime-8.0
```

### Command Explanation:

- The Microsoft package file adds Microsoft package sources to Ubuntu.
- dpkg -i installs the package source configuration.
- apt update refreshes packages again after adding Microsoft sources.
- dotnet-sdk-8.0 installs build tools and runtime.
- aspnetcore-runtime-8.0 installs the ASP.NET Core runtime used to run Web APIs.

### Expected Output / Result:

.NET SDK and ASP.NET Core runtime should install successfully.

### Verification:

Run dotnet --info to verify SDK and runtime details.

```

root@d0d317c574c2:/# wget https://packages.microsoft.com/config/ubuntu/24.04/packages-microsoft-prod.deb -O packages-microsoft-prod.deb
--2026-06-23 10:21:53-- https://packages.microsoft.com/config/ubuntu/24.04/packages-microsoft-prod.deb
Resolving packages.microsoft.com (packages.microsoft.com)... 13.107.246.58, 13.107.213.58, 13.107.246.58, ...
Connecting to packages.microsoft.com (packages.microsoft.com)|13.107.246.58|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 4288 (4.2K) [application/octet-stream]
Saving to: 'packages-microsoft-prod.deb'

packages-microsoft-prod.deb      100%[=====] 4.19K  --.-KB/s

2026-06-23 10:21:53 (2.66 GB/s) - 'packages-microsoft-prod.deb' saved [4288/4288]

root@d0d317c574c2:/# dpkg -i packages-microsoft-prod.deb
Selecting previously unselected package packages-microsoft-prod.
(Reading database ... 2428 files and directories currently installed.)
Preparing to unpack packages-microsoft-prod.deb ...
Unpacking packages-microsoft-prod (1.2-ubuntu24.04) ...
Setting up packages-microsoft-prod (1.2-ubuntu24.04) ...
root@d0d317c574c2:/# apt update
Get:1 https://packages.microsoft.com/ubuntu/24.04/prod noble InRelease [3600 B]
Get:2 https://packages.microsoft.com/ubuntu/24.04/prod noble/main arm64 Packages [151 kB]
Get:3 https://packages.microsoft.com/ubuntu/24.04/prod noble/main all Packages [643 B]
Get:4 https://packages.microsoft.com/ubuntu/24.04/prod noble/main amd64 Packages [181 kB]
Get:5 https://packages.microsoft.com/ubuntu/24.04/prod noble/main armhf Packages [11.6 kB]
Hit:6 http://ports.ubuntu.com/ubuntu-ports noble InRelease
Hit:7 http://ports.ubuntu.com/ubuntu-ports noble-updates InRelease
Hit:8 http://ports.ubuntu.com/ubuntu-ports noble-backports InRelease
Hit:9 http://ports.ubuntu.com/ubuntu-ports noble-security InRelease
Get:10 http://ports.ubuntu.com/ubuntu-ports noble/main arm64 Components [658 kB]
Get:11 http://ports.ubuntu.com/ubuntu-ports noble/universe arm64 Components [5467 kB]
Get:12 http://ports.ubuntu.com/ubuntu-ports noble/multiverse arm64 Components [37.7 kB]
Get:13 http://ports.ubuntu.com/ubuntu-ports noble-updates/main arm64 Components [239 kB]
Get:14 http://ports.ubuntu.com/ubuntu-ports noble-updates/universe arm64 Components [534 kB]
Get:15 http://ports.ubuntu.com/ubuntu-ports noble-updates/restricted arm64 Components [159 B]
Get:16 http://ports.ubuntu.com/ubuntu-ports noble-updates/multiverse arm64 Components [159 B]
Get:17 http://ports.ubuntu.com/ubuntu-ports noble-backports/main arm64 Components [3711 B]
Get:18 http://ports.ubuntu.com/ubuntu-ports noble-backports/universe arm64 Components [11.3 kB]
Get:19 http://ports.ubuntu.com/ubuntu-ports noble-backports/restricted arm64 Components [160 B]
Get:20 http://ports.ubuntu.com/ubuntu-ports noble-backports/multiverse arm64 Components [161 B]
Preparing to unpack .../06-dotnet-runtime-8.0.8.0.28-0ubuntu1-24.04.1_arm64.deb ...
Unpacking dotnet-runtime-8.0 (8.0.28-0ubuntu1-24.04.1) ...
Selecting previously unselected package aspnetcore-runtime-8.0.
Preparing to unpack .../07-aspnetcore-runtime-8.0.8.0.28-0ubuntu1-24.04.1_arm64.deb ...
Unpacking aspnetcore-runtime-8.0 (8.0.28-0ubuntu1-24.04.1) ...
Selecting previously unselected package aspnetcore-targeting-pack-8.0.
Preparing to unpack .../08-aspnetcore-targeting-pack-8.0.8.0.28-0ubuntu1-24.04.1_arm64.deb ...
Unpacking aspnetcore-targeting-pack-8.0 (8.0.28-0ubuntu1-24.04.1) ...
Selecting previously unselected package dotnet-apphost-pack-8.0.
Preparing to unpack .../09-dotnet-apphost-pack-8.0.8.0.28-0ubuntu1-24.04.1_arm64.deb ...
Unpacking dotnet-apphost-pack-8.0 (8.0.28-0ubuntu1-24.04.1) ...
Selecting previously unselected package dotnet-targeting-pack-8.0.
Preparing to unpack .../10-dotnet-targeting-pack-8.0.8.0.28-0ubuntu1-24.04.1_arm64.deb ...
Unpacking dotnet-targeting-pack-8.0 (8.0.28-0ubuntu1-24.04.1) ...
Selecting previously unselected package dotnet-templates-8.0.
Preparing to unpack .../11-dotnet-templates-8.0.8.0.128-0ubuntu1-24.04.1_arm64.deb ...
Unpacking dotnet-templates-8.0 (8.0.128-0ubuntu1-24.04.1) ...
Selecting previously unselected package netstandard-targeting-pack-2.1-8.0.
Preparing to unpack .../12-netstandard-targeting-pack-2.1-8.0.8.0.128-0ubuntu1-24.04.1_arm64.deb ...
Unpacking netstandard-targeting-pack-2.1-8.0 (8.0.128-0ubuntu1-24.04.1) ...
Selecting previously unselected package dotnet-sdk-8.0.
Preparing to unpack .../13-dotnet-sdk-8.0.8.0.128-0ubuntu1-24.04.1_arm64.deb ...
Unpacking dotnet-sdk-8.0 (8.0.128-0ubuntu1-24.04.1) ...
Setting up dotnet-targeting-pack-8.0 (8.0.28-0ubuntu1-24.04.1) ...
Setting up netstandard-targeting-pack-2.1-8.0 (8.0.128-0ubuntu1-24.04.1) ...
Setting up libltng-ust-common1t64:arm64 (2.13.7-1.iubuntu2) ...
Setting up aspnetcore-targeting-pack-8.0 (8.0.28-0ubuntu1-24.04.1) ...
Setting up dotnet-apphost-pack-8.0 (8.0.28-0ubuntu1-24.04.1) ...
Setting up libnuma1:arm64 (2.0.18-1ubuntu0.24.04.1) ...
Setting up libltng-ust-ct15t64:arm64 (2.13.7-1.iubuntu2) ...
Setting up dotnet-host-8.0 (8.0.28-0ubuntu1-24.04.1) ...
Setting up dotnet-hostfxr-8.0 (8.0.28-0ubuntu1-24.04.1) ...
Setting up dotnet-templates-8.0 (8.0.128-0ubuntu1-24.04.1) ...
Setting up libltng-ust1t64:arm64 (2.13.7-1.iubuntu2) ...
Setting up dotnet-runtime-8.0 (8.0.28-0ubuntu1-24.04.1) ...
Setting up aspnetcore-runtime-8.0 (8.0.28-0ubuntu1-24.04.1) ...
Setting up dotnet-sdk-8.0 (8.0.128-0ubuntu1-24.04.1) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
root@d0d317c574c2:/#

```

Student Notes: installed successfully

## Step 7: Verify .NET Installation

**Objective:** Check that .NET is installed correctly.

**Commands:**

```
dotnet --version
```

```
dotnet --info
```

**Command Explanation:**



Vaibhav Gupta (vgupta1@presidio.com)

- `dotnet --version` shows the installed SDK version.
- `dotnet --info` shows installed SDKs, runtimes, OS and architecture.

**Expected Output / Result:**

Expected output should show .NET 8.x SDK and Microsoft.AspNetCore.App 8.x runtime.

**Verification:**

Capture `dotnet --info` output.

Vaibhav Gupta (vgupta1@presidio.com)

```
[root@d0d317c574c2:/# dotnet --version
8.0.128
[root@d0d317c574c2:/# dotnet --info
.NET SDK:
  Version:           8.0.128
  Commit:            d091372eb2
  Workload version:  8.0.100-manifests.f89b5038

Runtime Environment:
  OS Name:            ubuntu
  OS Version:         24.04
  OS Platform:        Linux
  RID:                ubuntu.24.04-arm64
  Base Path:          /usr/lib/dotnet/sdk/8.0.128/

.NET workloads installed:
  Workload version: 8.0.100-manifests.f89b5038
  There are no installed workloads to display.

Host:
  Version:           8.0.28
  Architecture:      arm64
  Commit:            46295af582

.NET SDKs installed:
  8.0.128 [/usr/lib/dotnet/sdk]

.NET runtimes installed:
  Microsoft.AspNetCore.App 8.0.28 [/usr/lib/dotnet/shared/Microsoft.AspNetCore.App]
  Microsoft.NETCore.App 8.0.28 [/usr/lib/dotnet/shared/Microsoft.NETCore.App]

Other architectures found:
  None

Environment variables:
  Not set

global.json file:
  Not found

Learn more:
  https://aka.ms/dotnet/info

Download .NET:
  https://aka.ms/dotnet/download
root@d0d317c574c2:/# █
```

Student Notes: working fine

Vaibhav Gupta (vgupta1@presidio.com)

## Step 8: Install Node.js and npm

**Objective:** Install Node.js and npm so the Angular project can be built.

### Commands:

```
curl -fsSL https://deb.nodesource.com/setup_20.x | bash -  
  
apt install -y nodejs
```

### Command Explanation:

- NodeSource setup script adds the Node.js package repository.
- Node.js is required to run Angular build tools.
- npm is installed along with Node.js.

### Expected Output / Result:

Node.js and npm should be installed.

### Verification:

Run `node -v` and `npm -v`.

```
[root@d0d317c574c2:/# curl -fsSL https://deb.nodesource.com/setup_20.x | bash -
2026-06-23 10:26:34 - Installing pre-requisites
Hit:1 https://packages.microsoft.com/ubuntu/24.04/prod noble InRelease
Hit:2 http://ports.ubuntu.com/ubuntu-ports noble InRelease
Hit:3 http://ports.ubuntu.com/ubuntu-ports noble-updates InRelease
Hit:4 http://ports.ubuntu.com/ubuntu-ports noble-backports InRelease
Hit:5 http://ports.ubuntu.com/ubuntu-ports noble-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
2 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ca-certificates is already the newest version (20260601~24.04.1).
curl is already the newest version (8.5.0-2ubuntu10.9).
gnupg is already the newest version (2.4.4-2ubuntu17.4).
The following NEW packages will be installed:
  apt-transport-https
0 upgraded, 1 newly installed, 0 to remove and 2 not upgraded.
Need to get 3970 B of archives.
After this operation, 36.9 kB of additional disk space will be used.
Get:1 http://ports.ubuntu.com/ubuntu-ports noble-updates/universe arm64 apt-transport-https all 2.8.3 [3970 B]
Fetched 3970 B in 0s (19.7 kB/s)
debconf: delaying package configuration, since apt-utils is not installed
Selecting previously unselected package apt-transport-https.
(Reading database ... 18855 files and directories currently installed.)
Preparing to unpack .../apt-transport-https_2.8.3_all.deb ...
Unpacking apt-transport-https (2.8.3) ...
Setting up apt-transport-https (2.8.3) ...
Hit:1 https://packages.microsoft.com/ubuntu/24.04/prod noble InRelease
Get:2 https://deb.nodesource.com/node_20.x nodistro InRelease [12.1 kB]
Hit:3 http://ports.ubuntu.com/ubuntu-ports noble InRelease
Hit:4 http://ports.ubuntu.com/ubuntu-ports noble-updates InRelease
Get:5 https://deb.nodesource.com/node_20.x nodistro/main arm64 Packages [14.6 kB]
Hit:6 http://ports.ubuntu.com/ubuntu-ports noble-backports InRelease
Hit:7 http://ports.ubuntu.com/ubuntu-ports noble-security InRelease
Fetched 26.8 kB in 2s (15.9 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
2 packages can be upgraded. Run 'apt list --upgradable' to see them.
2026-06-23 10:26:44 - Repository configured successfully.
2026-06-23 10:26:44 - To install Node.js, run: apt install nodejs -y
2026-06-23 10:26:44 - You can use N|solid Runtime as a node.js alternative
2026-06-23 10:26:44 - To install N|solid Runtime, run: apt install nsolid -y

[root@d0d317c574c2:/# apt install -y nodejs
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  nodejs
0 upgraded, 1 newly installed, 0 to remove and 2 not upgraded.
Need to get 31.3 MB of archives.
After this operation, 196 MB of additional disk space will be used.
Setting up nodejs (20.20.2 nodesource) ...

[root@d0d317c574c2:/# node -v
v20.20.2
[root@d0d317c574c2:/# npm -v
10.8.2
root@d0d317c574c2:/#
```

Student Notes: installed

## Step 9: Verify Node.js and npm

**Objective:** Confirm Node.js and npm versions.

### Commands:

```
node -v
npm -v
```

### Command Explanation:

- node -v displays Node.js version.
- npm -v displays npm version.

### Expected Output / Result:

Expected Node.js version should be v20.x or compatible with your Angular project.

### Verification:

Capture both command outputs.

```
Setting up nodejs (20.20.2~nodesource1) ...
[root@d0d317c574c2:/# node -v
v20.20.2
[root@d0d317c574c2:/# npm -v
10.8.2
root@d0d317c574c2:/#
```

Student Notes: installed

## Step 10: Install Angular CLI

**Objective:** Install Angular command line tool globally.

### Commands:

```
npm install -g @angular/cli
ng version
```

### Command Explanation:

- npm install -g installs a package globally inside the container.
- @angular/cli provides the ng command.
- ng version verifies Angular CLI installation.

### Expected Output / Result:

Expected output shows Angular CLI version.

### Verification:



Capture ng version output.

```
root@d0d317c574c2:/#
```

Full-Stack Ubuntu Container Deployment Lab Manual | Screenshot Workbook

Vaibhav Gupta (vgupta1@presidio.com)  
Expected output: nginx version: nginx/x.x.x

#### Verification:

Capture nginx -v output.

```
Processing triggers for libc-bin (2.35-0ubuntu0.7) ...  
[root@d0d317c574c2:/# nginx -v  
nginx version: nginx/1.24.0 (Ubuntu)  
root@d0d317c574c2:/#
```

Student Notes: installed

### Step 12: Create Workspace Folder

**Objective:** Create a clean folder where both projects will be cloned.

#### Commands:

```
mkdir -p /workspace  
cd /workspace  
pwd
```

#### Command Explanation:

- mkdir -p creates the folder if it does not already exist.
- cd /workspace moves into the working folder.
- pwd confirms the current folder.

#### Expected Output / Result:

Expected output: /workspace

#### Verification:

Capture pwd output showing /workspace.

```
root@d0d317c574c2:/# mkdir -p /workspace  
cd /workspace  
pwd  
/workspace  
[root@d0d317c574c2:/workspace# pwd  
/workspace  
root@d0d317c574c2:/workspace#
```

Student Notes: created workspace folder

### Step 13: Clone .NET API Repository

**Objective:** Pull the backend source code from GitHub.

#### Commands:

Vaibhav Gupta (vgupta1@presidio.com)  
cd /workspace

```
git clone https://github.com/114-Vaibhav/demo-dotnet-api.git
```

```
ls
```

#### Command Explanation:

- Replace <DOTNET\_API\_REPO\_URL> with the real GitHub URL.
- git clone downloads the repository into a local folder.
- ls lists the cloned folder.

#### Expected Output / Result:

You should see the backend repository folder.

#### Verification:

Capture git clone success output and ls output.

```
bash: cd: demo-api: No such file or directory
[root@d0d317c574c2:/workspace# git clone https://github.com/114-Vaibhav/demo-dotnet-api.git
Cloning into 'demo-dotnet-api'...
remote: Enumerating objects: 15, done.
remote: Counting objects: 100% (15/15), done.
remote: Compressing objects: 100% (12/12), done.
Receiving objects: 100% (15/15), 9.03 KiB | 1.81 MiB/s, done.
Resolving deltas: 100% (2/2), done.
remote: Total 15 (delta 2), reused 15 (delta 2), pack-reused 0 (from 0)
[root@d0d317c574c2:/workspace# ls
demo-dotnet-api
root@d0d317c574c2:/workspace#
```

Student Notes: created and cloned dotnet demo

## Step 14: Restore and Build .NET API

**Objective:** Download NuGet packages and build the backend application.

#### Commands:

```
cd /workspace/ demo-dotnet-api
```

```
dotnet restore
```

```
dotnet build
```

#### Command Explanation:

- dotnet restore downloads NuGet dependencies.
- dotnet build compiles the application.
- Replace <DOTNET\_API\_FOLDER\_NAME> with the actual cloned folder name.

#### Expected Output / Result:

Vaibhav Gupta (vgupta1@presidio.com)

Expected output: Build succeeded.

#### Verification:

Capture Build succeeded output.

```
[root@d0d317c574c2:/workspace/demo-dotnet-api# dotnet restore
  Determining projects to restore...
  Restored /workspace/demo-dotnet-api/DemoApi.csproj (in 204 ms).
[root@d0d317c574c2:/workspace/demo-dotnet-api# dotnet build
MSBuild version 17.8.49+7806cbf7b for .NET
  Determining projects to restore...
  All projects are up-to-date for restore.
  DemoApi -> /workspace/demo-dotnet-api/bin/Debug/net8.0/DemoApi.dll

Build succeeded.
    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:01.97
root@d0d317c574c2:/workspace/demo-dotnet-api#
```

Student Notes: there was some issue of dotnet version after clone, build successful

### Step 15: Run .NET API for Initial Test

**Objective:** Start the backend API inside the container.

#### Commands:

```
cd /workspace/<DOTNET_API_FOLDER_NAME>

ASPNETCORE_URLS=http://0.0.0.0:5000 dotnet run
```

#### Command Explanation:

- ASPNETCORE\_URLS tells the API to listen on all network interfaces.
- http://0.0.0.0:5000 makes it accessible through Docker port mapping.
- dotnet run starts the application from source code.

#### Expected Output / Result:

Expected output includes Now listening on: http://0.0.0.0:5000.

#### Verification:

From the host browser, try <http://localhost:5000> or <http://localhost:5000/swagger> if Swagger exists. Stop the API with Ctrl+C after testing.

Vaibhav Gupta (vgupta1@presidio.com)

```
[root@d0d317c574c2:/workspace/demo-dotnet-api# ASPNETCORE_URLS=http://0.0.0.0:5000 dotnet run
Building...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5009
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: /workspace/demo-dotnet-api
```

Student Notes: running on 5009

## Step 16: Publish .NET API

**Objective:** Create a production-ready published output folder.

**Commands:**

```
cd /workspace/<DOTNET_API_FOLDER_NAME>
```

```
dotnet publish -c Release -o /app/api
```

**Command Explanation:**

- dotnet publish prepares compiled files for deployment.
- -c Release creates a release build.
- -o /app/api places the published output in /app/api.

**Expected Output / Result:**

Expected output says the project was published to /app/api.

**Verification:**

Run ls /app/api and capture the files.

```
[root@d0d317c574c2:/workspace/demo-dotnet-api# dotnet publish -c Release -o /app/api
MSBuild version 17.8.49+7806cbf7b for .NET
Determining projects to restore...
All projects are up-to-date for restore.
DemoApi -> /workspace/demo-dotnet-api/bin/Release/net8.0/DemoApi.dll
DemoApi -> /app/api/
[root@d0d317c574c2:/workspace/demo-dotnet-api# ls /app/api
DemoApi DemoApi.deps.json DemoApi.dll DemoApi.pdb DemoApi.runtimeconfig.json appsettings.Development.json appsettings.json web.config
[root@d0d317c574c2:/workspace/demo-dotnet-api#
```

## Step 17: Run Published API in Background

**Objective:** Run the published API as a background process so Nginx can proxy to it.



Vaibhav Gupta (vgupta1@presidio.com)

### Commands:

```
cd /app/api

ASPNETCORE_URLS=http://127.0.0.1:5000 nohup dotnet DemoApi.dll > /var/log/dotnet-api.log 2>&1 &

ps -ef | grep dotnet

cat /var/log/dotnet-api.log
```

### Command Explanation:

- nohup allows the process to continue running after the shell returns.
- & starts the process in the background.
- 127.0.0.1:5000 means the API listens locally inside the container.
- <DOTNET\_PROJECT\_FILE\_NAME>.dll must be replaced with your actual DLL name.
- The log file helps troubleshoot startup errors.

### Expected Output / Result:

Expected: ps output should show a dotnet process. The log should show the API started successfully.

### Verification:

Capture ps -ef and log output.

```
root@d0d317c574c2:/workspace/demo-dotnet-api# cd /app/api
root@d0d317c574c2:/app/api# ASPNETCORE_URLS=http://127.0.0.1:5000 nohup dotnet DemoApi.dll > /var/log/dotnet-api.log 2>&1 &

ps -ef | grep dotnet
[1] 6175
root      5890      1  2 12:04 pts/0    00:00:16 /usr/lib/dotnet/dotnet exec /usr/lib/dotnet/sdk/8.0.128/Roslyn/bincore/VBCSCompiler.dll -pipename
root      6175      1  99 12:15 pts/0    00:00:00 dotnet DemoApi.dll
root      6177      1  0 12:15 pts/0    00:00:00 grep --color=auto dotnet
root@d0d317c574c2:/app/api# cat /var/log/dotnet-api.log
nohup: ignoring input
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://127.0.0.1:5000
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Production
info: Microsoft.Hosting.Lifetime[0]
      Content root path: /app/api
root@d0d317c574c2:/app/api#

root@d0d317c574c2:/app/api# ps -ef
UID          PID    PPID  C STIME TTY          TIME CMD
root           1        0  0 10:13 pts/0    00:00:00 bash
root       5890      1  2 12:04 pts/0    00:00:16 /usr/lib/dotnet/dotnet exec /usr/lib/dotnet/sdk/8.0.128/Roslyn/bincore/VBCSCompiler.dll -pipename
root       6175      1  1 12:15 pts/0    00:00:00 dotnet DemoApi.dll
root       6210      1  99 12:16 pts/0    00:00:00 ps -ef
root@d0d317c574c2:/app/api#
```

Student Notes: running in background

## Step 18: Clone Angular Repository

**Objective:** Pull the Angular frontend source code from GitHub.

### Commands:

```
cd /workspace

git clone https://github.com/114-Vaibhav/demo-angular-app.git
```

Vaibhav Gupta (vgupta1@presidio.com)

```
ls
```

#### Command Explanation:

- Replace <ANGULAR\_REPO\_URL> with your real Angular GitHub repository URL.
- The repository will be downloaded into /workspace.
- ls confirms the folder exists.

#### Expected Output / Result:

You should see the Angular project folder.

#### Verification:

Capture git clone success output.

```
root@d0d317c574c2:/# cd /workspace
root@d0d317c574c2:/workspace# git clone https://github.com/114-Vaibhav/demo-angular-app.git
Cloning into 'demo-angular-app'...
remote: Enumerating objects: 31, done.
remote: Counting objects: 100% (31/31), done.
remote: Compressing objects: 100% (27/27), done.
remote: Total 31 (delta 1), reused 31 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (31/31), 79.55 KiB | 479.00 KiB/s, done.
Resolving deltas: 100% (1/1), done.
root@d0d317c574c2:/workspace# ls
demo-angular-app  demo-dotnet-api
root@d0d317c574c2:/workspace#
```

Student Notes: cloned

## Step 19: Install Angular Dependencies

**Objective:** Install packages required by the Angular project.

#### Commands:

```
cd /workspace/<ANGULAR_FOLDER_NAME>
```

```
npm install
```

#### Command Explanation:

- npm install reads package.json.
- It downloads project dependencies into node\_modules.
- This may take a few minutes.

#### Expected Output / Result:

Expected output should complete without fatal errors.

#### Verification:

Capture the final npm install success output.

#### Screenshot Placeholder

Vaibhav Gupta (vgupta1@presidio.com)

```
[root@d0d317c574c2:/workspace/demo-angular-app# npm i
changed 137 packages, and audited 467 packages in 10s

112 packages are looking for funding
  run `npm fund` for details

7 vulnerabilities (3 low, 4 high)

To address all issues possible (including breaking changes), run:
  npm audit fix --force

Some issues need review, and may require choosing
a different dependency.

Run `npm audit` for details.
root@d0d317c574c2:/workspace/demo-angular-app#
```

Student Notes: node\_modules installed

## Step 20: Configure Angular API URL Placeholder

**Objective:** Update Angular configuration so API calls go through Nginx /api path.

### Commands:

```
# Open your Angular environment file if your project uses one
nano src/environments/environment.ts

# Example value to use inside Angular
# apiBaseUrl: "/api"
```

### Command Explanation:

- This step depends on your Angular project structure.
- The recommended browser-facing API URL is /api.
- Using /api allows Nginx to forward calls to the backend API running on port 5000 inside the container.
- If your app uses a different config file, update that file instead.

### Expected Output / Result:

Angular source should point API calls to /api or relative API paths.

### Verification:

Capture the updated environment/configuration file.

```
EOF
root@d0d317c574c2:/workspace/demo-angular-app# cat > src/app/app.component.ts <<'EOF'
import { Component } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule],
  template: `
    <div style="text-align:center; margin-top: 50px; font-family: sans-serif;">
      <h1>Angular + .NET Full-Stack Demo</h1>
      <button (click)="getWeather()" style="padding: 10px 20px; font-size: 16px; cursor: pointer;">
        Fetch Weather from .NET API
      </button>
      <ul style="list-style: none; padding: 0; margin-top: 20px;">
        <li *ngFor="let w of forecasts" style="margin: 5px auto; padding: 10px; background: #eee; width: 300px; border-radius: 5px;">
          <strong>{{ w.date }}</strong>: {{ w.summary }} ({{ w.temperatureC }}\302\260C)
        </li>
      </ul>
    </div>
  `
})
export class AppComponent {
  forecasts: any[] = [];

  constructor(private http: HttpClient) {}

  getWeather() {
    // Notice we are using the relative path "/api/weatherforecast" exactly as Step 20 requested!
    this.http.get<any[]>('/api/weatherforecast').subscribe({
      next: (data) => this.forecasts = data,
      error: (err) => alert('Failed to connect to API. Check console for details.')
    });
  }
}
EOF
root@d0d317c574c2:/workspace/demo-angular-app# █
```

## Step 21: Build Angular Application

**Objective:** Build Angular static files for production.

### Commands:

```
cd /workspace/<ANGULAR_FOLDER_NAME>
```

```
ng build --configuration production
```

### Command Explanation:

- ng build compiles the Angular app.
- --configuration production creates optimized static files.
- The output usually goes into dist/<project-name>.

### Expected Output / Result:

Expected output: build completed successfully and dist folder created.

### Verification:

Run ls dist and capture the output.

```
root@d0d317c574c2:/workspace/demo-angular-app# ng build --configuration production

Would you like to share pseudonymous usage data about this project with the Angular Team
at Google under Google's Privacy Policy at https://policies.google.com/privacy. For more
details and how to change this setting, see https://angular.dev/cli/analytics.

[ ] No
Global setting: not set
Local setting: disabled
Effective status: disabled
Initial chunk files | Names | Raw size | Estimated transfer size
main-2G6K6LTV.js | main | 147.89 kB | 42.88 kB
styles-5INURTS0.css | styles | 0 bytes | 0 bytes
| Initial total | 147.89 kB | 42.88 kB

Application bundle generation complete. [9.753 seconds] - 2026-06-23T06:59:47.322Z

Output location: /workspace/demo-angular-app/dist/demo-angular-app

root@d0d317c574c2:/workspace/demo-angular-app#
```

Student Notes: build successful

## Step 22: Deploy Angular Build to Nginx Web Root

**Objective:** Copy the Angular build output into the Nginx hosting folder.

### Commands:

```
rm -rf /var/www/html/*

mkdir -p /var/www/html/angular

cp -r /workspace/demo-angular-app/dist/demo-angular-app/* /var/www/html/angular/

ls -la /var/www/html/angular
```

### Command Explanation:

- `rm -rf` clears default Nginx files.
- `mkdir` creates the folder that will host Angular files.
- `cp` copies built Angular files into Nginx web root.
- `<ANGULAR_DIST_FOLDER_NAME>` must match the actual folder inside dist.

### Expected Output / Result:

Expected output should include index.html and static JavaScript/CSS files.

### Verification:

Capture `ls -la /var/www/html/angular` output.

Vaibhav Gupta (vgupta1@presidio.com)

```
root@d0d317c574c2:/workspace/demo-angular-app# # 1. Clear out the incorrect copy
rm -rf /var/www/html/angular/*

# 2. Copy the files from inside the 'browser' folder this time
cp -r /workspace/demo-angular-app/dist/demo-angular-app/browser/* /var/www/html/angular/

# 3. Verify it worked
ls -la /var/www/html/angular
total 176
drwxr-xr-x 2 root root  4096 Jun 23 12:35 .
drwxr-xr-x 3 root root  4096 Jun 23 12:31 ..
-rw-r--r-- 1 root root 15086 Jun 23 12:35 favicon.ico
-rw-r--r-- 1 root root   428 Jun 23 12:35 index.html
-rw-r--r-- 1 root root 147887 Jun 23 12:35 main-2G6K6LTV.js
-rw-r--r-- 1 root root     0 Jun 23 12:35 styles-5INURTS0.css
root@d0d317c574c2:/workspace/demo-angular-app#
```

## Step 23: Create Nginx Configuration for Angular and API Reverse Proxy

**Objective:** Configure Nginx to serve Angular and forward API requests to the .NET backend.

### Commands:

```
cat > /etc/nginx/sites-available/default <<'EOF'
server {
    listen 80;
    server_name _;

    root /var/www/html/angular;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /api/ {
        proxy_pass http://127.0.0.1:5000/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
EOF

nginx -t
```

### Command Explanation:

- listen 80 tells Nginx to listen on port 80 inside the container.
- root points to Angular build files.
- try\_files supports Angular client-side routing.
- location /api forwards API requests to the .NET app on port 5000.
- nginx -t validates the configuration syntax.

### Expected Output / Result:

Expected output: syntax is ok and test is successful.

### Verification:



Vaibhav Gupta (vgupta1@presidio.com)

Capture nginx -t output.

```
root@d0d317c574c2:/workspace/demo-angular-app# cat > /etc/nginx/sites-available/default <<'EOF'
server {
    listen 80;
    server_name _;

    root /var/www/html/angular;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    location /api/ {
        proxy_pass http://127.0.0.1:5000/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
EOF
root@d0d317c574c2:/workspace/demo-angular-app# nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
root@d0d317c574c2:/workspace/demo-angular-app#
```

## Step 24: Start or Reload Nginx

**Objective:** Start Nginx so the Angular app becomes available.

### Commands:

```
nginx
```

# If Nginx is already running, use this instead:

```
nginx -s reload
```

```
ps -ef | grep nginx
```

### Command Explanation:

- nginx starts the web server.
- nginx -s reload reloads configuration without stopping Nginx.
- ps -ef confirms that Nginx master and worker processes are running.

### Expected Output / Result:

Expected output should show nginx master process and worker process.

### Verification:

Capture ps -ef | grep nginx output.

```
root@d0d317c574c2:/workspace/demo-angular-app# nginx
root@d0d317c574c2:/workspace/demo-angular-app# nginx -s reload
2026/06/23 12:38:11 [notice] 6429#6429: signal process started
root@d0d317c574c2:/workspace/demo-angular-app# ps -ef | grep nginx
root      6420      1  0 12:38 ?        00:00:00 nginx: master process nginx
www-data  6430    6420  0 12:38 ?        00:00:00 nginx: worker process
www-data  6431    6420  0 12:38 ?        00:00:00 nginx: worker process
www-data  6432    6420  0 12:38 ?        00:00:00 nginx: worker process
www-data  6433    6420  0 12:38 ?        00:00:00 nginx: worker process
www-data  6434    6420  0 12:38 ?        00:00:00 nginx: worker process
www-data  6435    6420  0 12:38 ?        00:00:00 nginx: worker process
www-data  6436    6420  0 12:38 ?        00:00:00 nginx: worker process
www-data  6437    6420  0 12:38 ?        00:00:00 nginx: worker process
root      6439      1  0 12:38 pts/0    00:00:00 grep --color=auto nginx
root@d0d317c574c2:/workspace/demo-angular-app#
```

## Step 25: Test from Inside the Container

**Objective:** Verify Angular and API proxy from inside Ubuntu first.

### Commands:

```
curl http://localhost

curl http://localhost/api/health

# If your API does not have /health, test another endpoint:
# curl http://localhost/api/<your-api-endpoint>
```

### Command Explanation:

- `curl http://localhost` checks Angular/Nginx.
- `curl http://localhost/api/health` checks whether Nginx can reach the .NET API.
- If your API does not have `/health`, use an endpoint from your backend project.

### Expected Output / Result:

Angular curl may return HTML. API curl should return API response.

### Verification:

Capture both curl outputs.

```
root@d0d317c574c2:/workspace/demo-angular-app# curl http://localhost
<!doctype html>
<html lang="en" data-beasties-container>
<head>
  <meta charset="utf-8">
  <title>DemoAngularApp</title>
  <base href="/">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="icon" type="image/x-icon" href="favicon.ico">
  <link rel="stylesheet" href="styles-5INURTS0.css"></head>
<body>
  <app-root></app-root>
<script src="main-2G6K6lTV.js" type="module"></script></body>
</html>
root@d0d317c574c2:/workspace/demo-angular-app# curl http://localhost/api/health
root@d0d317c574c2:/workspace/demo-angular-app# curl -i http://localhost/api/health
HTTP/1.1 404 Not Found
Server: nginx/1.24.0 (Ubuntu)
Date: Tue, 23 Jun 2026 07:11:49 GMT
Content-Length: 0
Connection: keep-alive

root@d0d317c574c2:/workspace/demo-angular-app# curl -i http://localhost/api/weatherforecast
HTTP/1.1 200 OK
Server: nginx/1.24.0 (Ubuntu)
Date: Tue, 23 Jun 2026 07:12:10 GMT
Content-Type: application/json; charset=utf-8
Transfer-Encoding: chunked
Connection: keep-alive

[{"date": "2026-06-24", "temperatureC": 33, "summary": "Hot", "temperatureF": 91}, {"date": "2026-06-25", "temperatureC": 28, "summary": "Warm", "temperatureF": 82}, {"date": "2026-06-26", "temperatureC": 44, "summary": "Cool", "temperatureF": 111}, {"date": "2026-06-27", "temperatureC": 18, "summary": "Hot", "temperatureF": 64}, {"date": "2026-06-28", "temperatureC": 15, "summary": "Hot", "temperatureF": 58}]root@d0d317c574c2:/workspace/d
emo-angular-app#
```

Vaibhav Gupta (vgupta1@presidio.com)

## Step 26: Test from Local Host Browser

**Objective:** Access the Angular app from the Windows browser.

### Commands:

Open browser on the host machine:

```
http://localhost:8080
```

Optional direct API test:

```
http://localhost:8080/api/health
```

### Command Explanation:

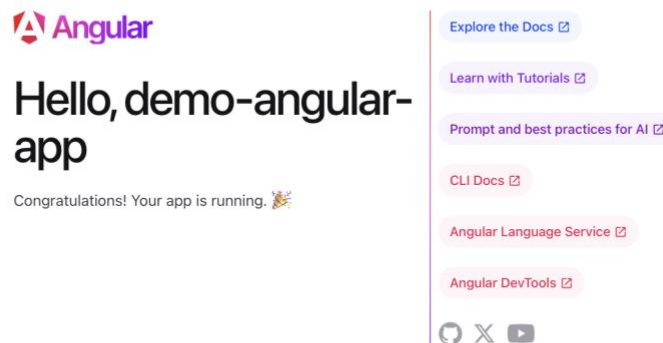
- Docker maps host port 8080 to container port 80.
- The browser talks to Nginx.
- Nginx serves Angular and proxies API calls.

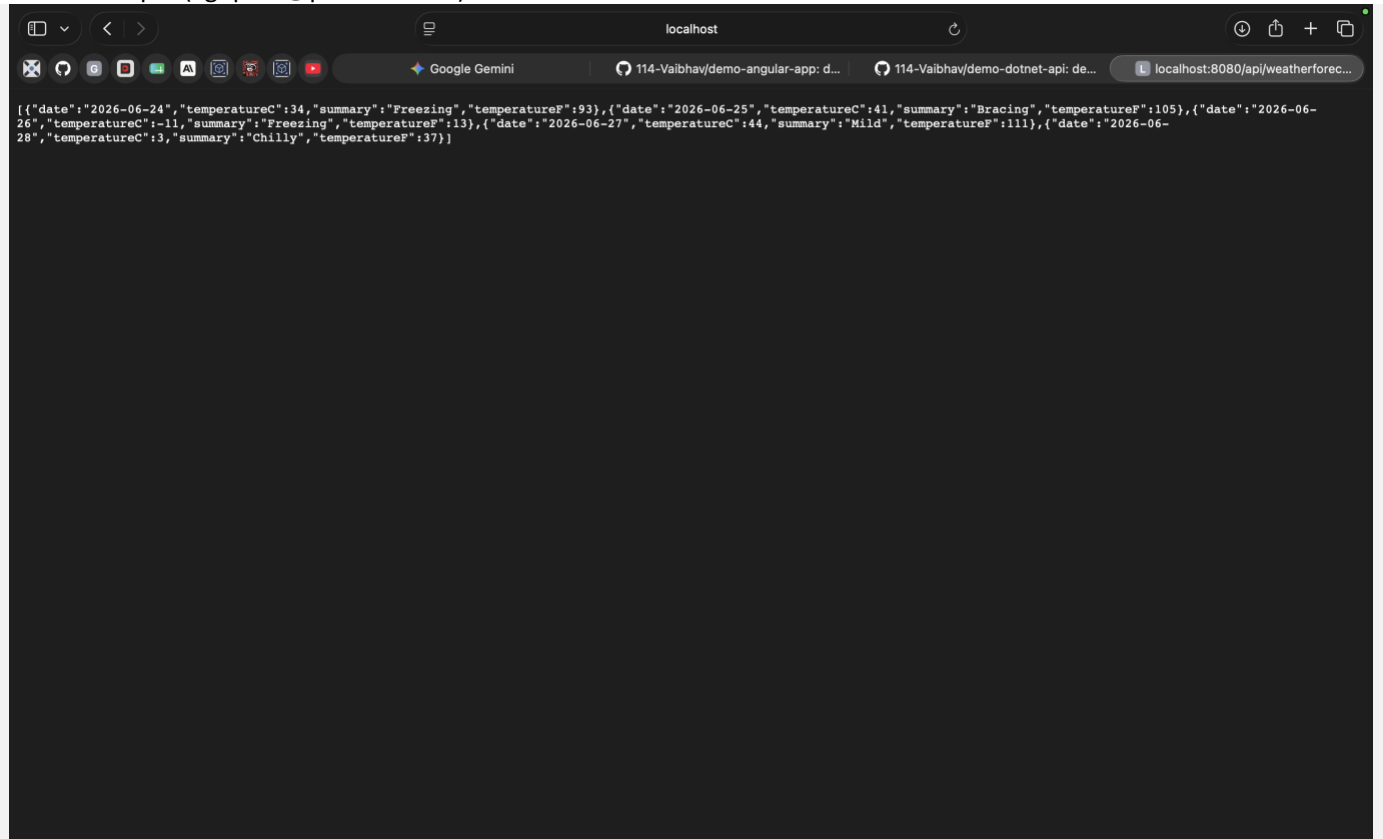
### Expected Output / Result:

Expected: Angular app opens in browser. API requests from Angular should reach the .NET backend.

### Verification:

Capture browser screenshot showing Angular loaded from localhost:8080.





## Step 27: Check Logs for Troubleshooting

**Objective:** Use logs to diagnose application or Nginx errors.

### Commands:

```
cat /var/log/dotnet-api.log
```

```
tail -n 50 /var/log/nginx/access.log
```

```
tail -n 50 /var/log/nginx/error.log
```

### Command Explanation:

- dotnet-api.log shows backend startup/runtime logs.
- Nginx access log shows browser/API requests.
- Nginx error log shows configuration and proxy errors.

### Expected Output / Result:

Expected: no fatal errors in logs.

### Verification:

Vaibhav Gupta (vgupta1@presidio.com)  
Capture logs if troubleshooting is required.

```
demo-angular-app#  
root@0d0d317c574c2:/workspace/demo-angular-app# cat /var/log/dotnet-api.log  
nohup: ignoring input  
info: Microsoft.Hosting.Lifetime[14]  
Now listening on: http://127.0.0.1:5000  
info: Microsoft.Hosting.Lifetime[0]  
Application started. Press Ctrl+C to shut down.  
info: Microsoft.Hosting.Lifetime[0]  
Hosting environment: Production  
info: Microsoft.Hosting.Lifetime[0]  
Content root path: /app/api  
warn: Microsoft.AspNetCore.HttpsPolicy.HttpsRedirectionMiddleware[3]  
Failed to determine the https port for redirect.  
root@0d0d317c574c2:/workspace/demo-angular-app# tail -n 50 /var/log/nginx/access.log  
127.0.0.1 - - [23/Jun/2026:12:39:47 +0530] "GET / HTTP/1.1" 200 428 "-" "curl/8.5.0"  
127.0.0.1 - - [23/Jun/2026:12:39:59 +0530] "GET /api/health HTTP/1.1" 404 0 "-" "curl/8.5.0"  
127.0.0.1 - - [23/Jun/2026:12:41:49 +0530] "GET /api/health HTTP/1.1" 404 0 "-" "curl/8.5.0"  
127.0.0.1 - - [23/Jun/2026:12:42:10 +0530] "GET /api/forecast HTTP/1.1" 200 386 "-" "curl/8.5.0"  
192.168.65.1 - - [23/Jun/2026:12:46:20 +0530] "GET / HTTP/1.1" 200 308 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/26.5 Safari/605.1.15"  
192.168.65.1 - - [23/Jun/2026:12:46:20 +0530] "GET /styles-5INJRTSO.css HTTP/1.1" 200 0 "http://localhost:8080/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/26.5 Safari/605.1.15"  
192.168.65.1 - - [23/Jun/2026:12:46:20 +0530] "GET /main-206K6LTV.js HTTP/1.1" 200 147887 "http://localhost:8080/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/26.5 Safari/605.1.15"  
192.168.65.1 - - [23/Jun/2026:12:46:20 +0530] "GET /apple-touch-icon-precomposed.png HTTP/1.1" 200 308 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Safari/605.1.15"  
192.168.65.1 - - [23/Jun/2026:12:46:21 +0530] "GET /apple-touch-icon.png HTTP/1.1" 200 308 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Safari/605.1.15"  
192.168.65.1 - - [23/Jun/2026:12:46:21 +0530] "GET /favicon.ico HTTP/1.1" 200 15086 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Safari/605.1.15"  
192.168.65.1 - - [23/Jun/2026:12:46:57 +0530] "GET /api/health HTTP/1.1" 404 0 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/26.5 Safari/605.1.15"  
192.168.65.1 - - [23/Jun/2026:12:47:06 +0530] "GET /api/forecast HTTP/1.1" 200 403 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/26.5 Safari/605.1.15"  
root@0d0d317c574c2:/workspace/demo-angular-app# tail -n 50 /var/log/nginx/error.log  
root@0d0d317c574c2:/workspace/demo-angular-app#
```

## Step 28: Stop and Restart Existing Container

**Objective:** Learn how to leave and return to the container without losing installed tools.

### Commands:

```
# Inside container  
exit  
  
# On host machine  
docker ps -a  
  
docker start fullstack-ubuntu  
  
docker exec -it fullstack-ubuntu bash
```

### Command Explanation:

- exit leaves the container shell.
- docker ps -a shows stopped and running containers.
- docker start restarts the existing container.
- docker exec opens a new shell inside the running container.

### Expected Output / Result:

You should return to the Ubuntu shell with installed tools still available.

### Verification:

Capture docker ps -a and shell prompt after docker exec.

```

root@d0d317c574c2:/workspace/demo-angular-app# exit
exit
[vaibhavgupta@FVF6905WQ085F-vaibhavgupta Day 42 June 23 % docker ps -a
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS        NAMES
d0d317c574c2   ubuntu:24.04                        "bash"                  9 hours ago   Exited (0) 9 seconds ago           fullstack-ubuntu
4b245ad1812e   ubuntu-nginx:v1                    "nginx -g 'daemon of..." 24 hours ago   Exited (0) 23 hours ago           ubuntu-nginx-web
dc7b9bb5a26b   ubuntu:24.04                        "bash"                  24 hours ago   Exited (127) 24 hours ago         ubuntu-nginx
0dc7ce06419c   nginx                               "/docker-entrypoint..." 24 hours ago   Exited (0) 23 hours ago           myweb
5f7ddcef7de8   ubuntu:24.04                        "bash"                  26 hours ago   Exited (0) 26 hours ago           competent_wescoff
a2254fc82da7   alpine                             "sh"                   26 hours ago   Exited (0) 26 hours ago           epic_goldberg
13079ad22f80   ubuntu:24.04                        "bash"                  27 hours ago   Exited (127) 26 hours ago         priceless_gagarin
09e5a352b86e   compose-postgres-api-lab-api       "dotnet CustomerApi..." 5 days ago    Exited (0) 4 days ago             compose-customer-api
3ab6a64fb993   postgres:16                         "docker-entrypoint.s..." 5 days ago    Exited (0) 4 days ago             compose-postgres-db
bffe5302e358   postgres:16                         "docker-entrypoint.s..." 5 days ago    Exited (0) 4 days ago             compose-lab-db-1
c99ba46aea13   dpag/pgadmin4:latest              "/entrypoint.sh"        5 days ago    Exited (137) 4 days ago           compose-lab-pgadmin-1
c59382637d34   customer-api:1.0                   "dotnet CustomerApi..." 6 days ago    Exited (0) 6 days ago             customer-api
120348d37874   postgres:16                         "docker-entrypoint.s..." 6 days ago    Exited (0) 6 days ago             postgres-db
d3504c56a347   postgres:latest                    "docker-entrypoint.s..." 7 days ago    Exited (1) 7 days ago             dreamy_solomon
3970431fe05b   postgres                            "docker-entrypoint.s..." 7 days ago    Exited (0) 7 days ago             some-postgres
[vaibhavgupta@FVF6905WQ085F-vaibhavgupta Day 42 June 23 % docker start fullstack-ubuntu
fullstack-ubuntu
[vaibhavgupta@FVF6905WQ085F-vaibhavgupta Day 42 June 23 % docker exec -it fullstack-ubuntu bash
root@d0d317c574c2:/#

```

## Step 29: Optional: Save the Configured Container as an Image

**Objective:** Convert the configured container into a reusable image for later labs.

### Commands:

```
# Run this on host machine after exiting the container if needed
```

```
docker commit fullstack-ubuntu fullstack-ubuntu-web:v1
```

```
docker images
```

### Command Explanation:

- `docker commit` saves the current container state as an image.
- This is useful for training demos.
- In production, a Dockerfile is preferred.

### Expected Output / Result:

Expected: image `fullstack-ubuntu-web:v1` appears in `docker images`.

### Verification:

Capture docker images output.

```

vaibhavgupta@FVF6905WQ085F-vaibhavgupta Day 42 June 23 % docker commit fullstack-ubuntu fullstack-ubuntu-web:v1
sha256:b677530e5f6a4c9f120e58ed3e826a33bda6d7488692a0c04ab96be9785c8e40
[vaibhavgupta@FVF6905WQ085F-vaibhavgupta Day 42 June 23 % docker images

```

| IMAGE                               | ID           | DISK USAGE | CONTENT SIZE | EXTRA |
|-------------------------------------|--------------|------------|--------------|-------|
| alpine:latest                       | 28bd5fe8b56d | 13.6MB     | 4.27MB       | U     |
| compose-postgres-api-lab-api:latest | aebc9500f8bf | 387MB      | 97MB         | U     |
| customer-api:1.0                    | 62593972ba26 | 350MB      | 88.3MB       | U     |
| dpag/pgadmin4:latest                | cefc4cc6b7d9 | 765MB      | 175MB        | U     |
| fullstack-ubuntu-web:v1             | 2e2b09bcad23 | 2.41GB     | 562MB        |       |
| nginx:latest                        | 42f2d2f4e18d | 259MB      | 64.3MB       | U     |
| node:22-alpine                      | 658326d8d4k1 | 229MB      | 58.6MB       |       |
| postgres:16                         | 081f1bc7bd5e | 663MB      | 165MB        | U     |
| postgres:latest                     | 29ee7bb30d00 | 671MB      | 167MB        | U     |
| ubuntu-nginx:v1                     | 588195ce8e26 | 240MB      | 68.2MB       | U     |
| ubuntu:24.04                        | 786a8b558f7b | 141MB      | 30.8MB       | U     |
| weather-ui:1.0                      | 399165a2f358 | 76.1MB     | 21.9MB       |       |
| weatherapi:1.0                      | aefb12c8db89 | 351MB      | 88.4MB       |       |
| weatherapi:single                   | 8cf0eb18e30e | 1.25GB     | 311MB        |       |

```

vaibhavgupta@FVF6905WQ085F-vaibhavgupta Day 42 June 23 %

```



Vaibhav Gupta (vgupta1@presidio.com)

## Step 30: Cleanup Lab Resources

**Objective:** Remove the lab container when finished.

### Commands:

```
docker stop fullstack-ubuntu
```

```
docker rm fullstack-ubuntu
```

```
# Optional: remove saved image
```

```
# docker rmi fullstack-ubuntu-web:v1
```

### Command Explanation:

- `docker stop` stops the container.
- `docker rm` removes the container.
- `docker rmi` removes the saved image if created.

### Expected Output / Result:

Container should be removed successfully.

### Verification:

Capture `docker ps -a` output after removal.



```
medtheopr.single 8c0e0de3ee 1.250s 311%
[vaibhavgupta@FVF6905WQ05F-vaibhavgupta Day 42 June 23 % docker stop fullstack-ubuntu
fullstack-ubuntu
[vaibhavgupta@FVF6905WQ05F-vaibhavgupta Day 42 June 23 % docker rm fullstack-ubuntu
fullstack-ubuntu
[vaibhavgupta@FVF6905WQ05F-vaibhavgupta Day 42 June 23 % ]
```

## Troubleshooting Guide

| Problem                                              | Fix                                                                                                                |
|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Port 8080 already in use                             | Use a different host port, for example -p 8090:80, then open <code>http://localhost:8090</code> .                  |
| dotnet command not found                             | Check whether .NET installation completed. Run <code>apt update</code> and reinstall <code>dotnet-sdk-8.0</code> . |
| ng command not found                                 | Angular CLI may not be installed globally. Run <code>npm install -g @angular/cli</code> .                          |
| Angular build fails                                  | Run <code>npm install</code> again. Check Node.js version compatibility. Read the error message carefully.         |
| Nginx configuration test fails                       | Run <code>nginx -t</code> and fix the exact line mentioned in the error.                                           |
| API works directly on port 5000 but not through /api | Check <code>proxy_pass</code> configuration and verify the API process is running on <code>127.0.0.1:5000</code> . |
| Browser shows Nginx default page                     | Make sure Angular files are copied to <code>/var/www/html/angular</code> and root points to that folder.           |
| Angular refresh shows 404                            | Make sure <code>try_files \$uri \$uri/ /index.html;</code> is present in the Nginx location / block.               |

## Final Lab Checklist

- ☐ Ubuntu container started with port mapping 8080:80 and 5000:5000. 
- ☐ Git installed and verified. 
- ☐ .NET SDK/runtime installed and verified. 
- ☐ Node.js and npm installed and verified. 
- ☐ Angular CLI installed and verified. 
- ☐ Nginx installed and verified. 
- ☐ .NET API repository cloned from GitHub. 
- ☐ .NET API restored, built, published, and running. 
- ☐ Angular repository cloned from GitHub. 
- ☐ Angular dependencies installed and production build completed. 
- ☐ Angular files copied to Nginx root folder. 
- ☐ Nginx reverse proxy configured for /api. 
- ☐ Angular app accessible from host browser at <http://localhost:8080>. 
- ☐ Angular can call backend API through Nginx proxy. 

## Knowledge Check Questions

1. Why do we map host port 8080 to container port 80?

Answer: So we can view the container's website from our own computer's browser.

2. What is the purpose of Git in this lab?

Answer: \_to download the projects code from the internet

---

3. Why do we install .NET SDK instead of only .NET runtime?

Answer: because we need the sdk to build the code not just run it.

---

4. What does npm install do?

Answer: it downloads all extra packages, store them in node\_modules

---

5. What does ng build --configuration production create?

Vaibhav Gupta (vgupta1@presidio.com)

Answer: it builds the final, ready to use website

---

6. Why does Angular need Nginx after build?

Answer: angular only creates the files, Nginx is the server needed to show them on internet

---

7. What is the role of proxy\_pass in Nginx?

Answer: it takes specific web requests and sends them to the backend api

---

8. Why do we use /api as the backend path?

Answer: to help Nginx easily tell the difference b/w website pages and backend api requests

---

9. What is the purpose of nohup when running the .NET API?

Answer: it keeps the api running in the background even if we close the command window

---

10. Why is Dockerfile preferred over docker commit in real projects?

Answer: it is easy to share and reuse, while commit is manual save.

---

## Appendix A: Placeholder Replacement Guide

|                            |                                                                   |
|----------------------------|-------------------------------------------------------------------|
| <DOTNET_API_REPO_URL>      | Example: https://github.com/your-org/your-api.git                 |
| <DOTNET_API_FOLDER_NAME>   | Example: your-api                                                 |
| <DOTNET_PROJECT_FILE_NAME> | Example: YourApi                                                  |
| <ANGULAR_REPO_URL>         | Example: https://github.com/your-org/your-angular-app.git         |
| <ANGULAR_FOLDER_NAME>      | Example: your-angular-app                                         |
| <ANGULAR_DIST_FOLDER_NAME> | Example: your-angular-app or browser depending on Angular version |

## Appendix B: Full Command Summary

```
docker pull ubuntu:24.04
```

```
docker run -it --name fullstack-ubuntu -p 8080:80 -p 5000:5000 ubuntu:24.04 bash
```

```
apt update
apt install -y git curl wget nano ca-certificates gnupg lsb-release software-properties-common
wget https://packages.microsoft.com/config/ubuntu/24.04/packages-microsoft-prod.deb -O packages-
microsoft-prod.deb
dpkg -i packages-microsoft-prod.deb
apt update
apt install -y dotnet-sdk-8.0 aspnetcore-runtime-8.0
curl -fsSL https://deb.nodesource.com/setup_20.x | bash -
apt install -y nodejs
npm install -g @angular/cli
apt install -y nginx
```

```
mkdir -p /workspace
cd /workspace
git clone <DOTNET_API_REPO_URL>
cd /workspace/<DOTNET_API_FOLDER_NAME>
dotnet restore
dotnet build
dotnet publish -c Release -o /app/api
cd /app/api
ASPNETCORE_URLS=http://127.0.0.1:5000 nohup dotnet <DOTNET_PROJECT_FILE_NAME>.dll >
/var/log/dotnet-api.log 2>&1 &
```

```
cd /workspace
git clone <ANGULAR_REPO_URL>
cd /workspace/<ANGULAR_FOLDER_NAME>
npm install
ng build --configuration production
rm -rf /var/www/html/*
mkdir -p /var/www/html/angular
cp -r /workspace/<ANGULAR_FOLDER_NAME>/dist/<ANGULAR_DIST_FOLDER_NAME>/* /var/www/html/angular/
```

```
nginx -t
nginx
curl http://localhost
curl http://localhost/api/health
```

```
# Host browser
# http://localhost:8080
```